# Exportación e inferencia

## Tiempo de exposición

35 segundos $\approx$ 82 palabras a 140 palabras/minuto.

- El modelo entrenado se exporta a ONNX.
- Exportación realizada desde el notebook de entrenamiento.
- En runtime se carga desde `socket_vision/models/line_segmentation`.
- El sistema ejecuta el modelo ONNX con OpenCV DNN.
- Cada frame se normaliza, pasa por la red y se convierte en máscaras binarias.

# Procesado de máscaras

## Tiempo de exposición

35 segundos $\approx$ 82 palabras a 140 palabras/minuto.

- Archivo: `socket_vision/vision/perception/line_segmentation.py`.
- Se aplica sigmoide y umbral para obtener las clases.
- La máscara del circuito se limpia con operaciones morfológicas.
- Se filtran componentes pequeños para reducir ruido.
- Se calcula el centro de la zona transitable en una región de interés.

# Lógica de seguimiento del circuito

## Tiempo de exposición

30 segundos $\approx$ 70 palabras a 140 palabras/minuto.

- Se compara el centro del circuito con el centro de la imagen.
- Si el circuito aparece desplazado, se genera un error lateral.
- Ese error se transforma en *steering*.
- Si el coche está centrado, mantiene avance recto.
- Si se pierde el circuito, puede pararse por seguridad.

# Esquiva de cajas

## Tiempo de exposición

40 segundos $\approx$ 93 palabras a 140 palabras/minuto.

- Archivo: `socket_vision/vision/runtime/autonomy.py`.
- Se analiza la máscara de cajas en la zona frontal.
- Se calcula posición, área y urgencia del obstáculo.
- Caja a la izquierda: giro hacia la derecha.
- Caja a la derecha: giro hacia la izquierda.
- Si el riesgo es alto, se reduce velocidad o se detiene.

# Control del coche en Jetson

**Tiempo de exposición**

40 segundos $\approx$ 93 palabras a 140 palabras/minuto.

- Archivo: `socket_vision/jetson/control_server.py`.
- Recibe comandos `auto_drive` por socket.
- Aplica *steering* y *throttle* con `NvidiaRacecar`.
- Incluye calibración de dirección y límites de seguridad.
- Permite control manual con prioridad temporal sobre el automático.
- Tiene watchdog para detener el coche si se pierden comandos.

# Arquitectura general

## Tiempo de exposición

35 segundos $\approx$ 82 palabras a 140 palabras/minuto.

- `circuito_segmentation`: dataset, entrenamiento, pruebas y exportación.
- `socket_vision`: visión en tiempo real, control, comunicación y Jetson.
- El PC se usa para entrenamiento y preparación.
- La Jetson ejecuta captura, comunicación y control del robot.
- El script `main.sh` sincroniza y lanza el flujo completo.

# Resultados de prueba real

## Tiempo de exposición

30 segundos $\approx$ 70 palabras a 140 palabras/minuto.

- Se validó la integración completa de visión, comunicación y control.
- El sistema segmenta circuito y cajas en la prueba real.
- Calcula el centro de avance y genera comandos de dirección y velocidad.
- Límite observado: campo de visión reducido o cámara demasiado adelantada.
- Límite observado: latencia por enviar vídeo al PC y devolver comandos a Jetson.

# Conclusiones

## Tiempo de exposición

40 segundos $\approx$ 93 palabras a 140 palabras/minuto.

- La segmentación semántica es adecuada porque da información por píxel.
- Dos clases son suficientes para la tarea: circuito y caja.
- La U-Net ligera permite trabajar con restricciones de cómputo.
- La lógica de control convierte máscaras en comandos simples y efectivos.
- Mejoras clave: cámara/posición con más campo de visión y reducción de latencia.
- Futuro: más datos, más luz, ajuste de umbrales, DeepLabv3+ o TensorRT.

# Cierre

## Tiempo de exposición

15 segundos $\approx$ 35 palabras a 140 palabras/minuto.

- Se construyó una tubería completa desde captura y etiquetado hasta conducción autónoma.
- El resultado no es solo un modelo entrenado: es una integración completa con Jetson y control del coche.